Study Plan — 6 Items a Day (Monday to Friday)

318 items over 53 study days in 11 weeks (138 topics, 62 practice, 18 Build Task, 100 project). Each day is 6 items (about 1-2 hours each). The weeks are an order, not fixed dates - go at your own pace. Do a topic's practice yourself before looking at the hints, then the solution last. Projects start right after the document that unlocks them.

Generated by `tools/build_study_plan.py` from the documents - do not edit by hand.

Summary

Week	What you study
Week 1	Doc01, Doc02, Doc03, Doc04
Week 2	Doc04, Project 1, Project 1.1, Doc05
Week 3	Doc05, Project 6, Project 6.1, Doc06
Week 4	Doc06, Project 7, Project 7.1, Doc07, Project 2
Week 5	Project 2, Project 2.1, Project 8, Project 8.1, Doc08, Project 9, Project 9.1
Week 6	Doc08b, Doc09, Project 10, Project 10.1, Doc10
Week 7	Doc10, Project 3, Project 3.1, Doc11
Week 8	Doc11, Project 4, Project 4.1, Project 11, Project 11.1, Project 12, Project 12.1, Doc12
Week 9	Doc12, Doc13, Doc14, Doc15, Doc16, Doc17
Week 10	Doc17, Doc18, Doc19, Project 5, Project 5.1, Project 13
Week 11	Project 13.1, Doc20

Week 1

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Doc01** · Functions, types, and why they matter	**Doc01** · Errors: a clean way to say "something specific	**Doc01** · Virtual environments: keeping projects separate	**Doc01** · `.env` files: keeping secrets out of your code	**Doc01** · Logging: better than `print()`	**Doc01 practice (1 of 4)** · Basic — typed function + custom error · Basic (part 2) —

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
		went wrong"				a real venv, start to finish
Tue	**Doc01 practice (2 of 4)** · Intermediate — read `.env` by hand, then explain why not to · Intermediate (part 2) — a logger with two output levels	**Doc01 practice (3 of 4)** · Real-world — wire config + logging together · Edge cases — is an empty value "missing" or "valid"?	**Doc01 practice (4 of 4)** · Failure handling — three different errors, three different reactions	**Doc01 Build Task (part 1 of 2)** · Build it: Config & Logging Foundation	**Doc01 Build Task (part 2 of 2)** · Finish and check it against the Test Cases	**Doc02** · HTTP: a request gets a response
Wed	**Doc02** · Timeouts: the failure that never tells you it's happening	**Doc02** · Retrying with exponential backoff and jitter	**Doc02** · JSON: two different kinds of "wrong"	**Doc02 practice (1 of 3)** · Basic — your first real API call · Intermediate — timeouts and retry-with-backoff	**Doc02 practice (2 of 3)** · Real-world — one reusable session instead of repeated headers · Edge cases — 200 doesn't mean safe to trust	**Doc02 practice (3 of 3)** · Failure — respecting a real rate limit
Thu	**Doc02 Build Task (part 1 of 2)** · Build it: HTTP Client Wrapper	**Doc02 Build Task (part 2 of 2)** · Finish and check it against the Test Cases	**Doc03** · Tokens: what the model actually reads	**Doc03** · Context window: a shared budget, not just "how much you can paste in"	**Doc03** · No memory: there's nothing "remembered" inside the model	**Doc03** · Randomness: `temperature` and `top_p`
Fri	**Doc03** · Cost: input and output tokens are priced differently	**Doc03** · Making things up: not a bug, just how	**Doc03 practice (1 of 3)** · Basic — guess token counts, then check ·	**Doc03 practice (2 of 3)** · Real-world — how long can this	**Doc03 practice (3 of 3)** · Failure — make it confidently	**Doc04** · The client, and the list of messages

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
		the model works	Intermediate — temperature and cost, side by side	conversation actually go? · Edge cases — where the ¾-word rule breaks	wrong, on purpose	

Week 2

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Doc04** · Streaming vs. waiting for the full reply	**Doc04** · Structured output: how "give me JSON" is actually guaranteed	**Doc04** · Error types, and which Doc02 rules apply to each	**Doc04** · Chat Completions vs. the newer Responses API	**Doc04 practice (1 of 3)** · Basic — your first real API call · Intermediate — build real memory	**Doc04 practice (2 of 3)** · Real-world — make it feel alive with streaming · Structured output — a checked answer, not a hope
Tue	**Doc04 practice (3 of 3)** · Edge cases — what actually happens when you go over the limit · Failure — a bad key, and a cost comparison	**Project 1** · SupportDesk (Concierge & Triage) — Setup (plan it first: the 7 steps)	**Project 1** · Build How testing works in this project (part 1 of 2)	**Project 1** · Test and finish How testing works in this project (part 2 of 2)	**Project 1** · Build Step 1 — A Script That Sends One Message and Prints the Reply (part 1 of 2)	**Project 1** · Test and finish Step 1 (part 2 of 2)

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Wed	**Project 1** · Build Step 2 — A Real Terminal Chat With Memory and Live Streaming (part 1 of 2)	**Project 1** · Test and finish Step 2 (part 2 of 2)	**Project 1** · Build Step 3 — A Triage Agent That Extracts a Structured Ticket From Free Text (part 1 of 2)	**Project 1** · Test and finish Step 3 (part 2 of 2)	**Project 1** · Build Step 4 — A Router That Sends Each Message to the Right Agent Automatically (final: 2 agents) (part 1 of 2)	**Project 1** · Test and finish Step 4 (part 2 of 2), then go through the Checklist Before You Call This Done
Thu	**Project 1.1 (practice twin)** · Build RepairDesk — Device-Repair Chat & Ticket-Intake Assistant from its story, then check the solution	**Doc05** · What a library gives you, and what it costs	**Doc05** · `ChatOpenAI`: a wrapper, not a different model	**Doc05** · Prompt templates: keeping the fixed part separate from the changing part	**Doc05** · Few-shot prompting: teaching the model by example	**Doc05** · LCEL: chaining pieces together with `|`
Fri	**Doc05** · Output parsers: turning raw text into a real Python object	**Doc05** · The three LangChain packages: `langchain-core`, `langchain-openai`, `langchain`	**Doc05 practice (1 of 4)** · Basic — your first LCEL chain · Intermediate — swap in a structured parser	**Doc05 practice (2 of 4)** · Real-world — rebuild Doc04's extraction, LCEL-style · Real-world (part 2) — run in parallel, fall back, and stream	**Doc05 practice (3 of 4)** · Real-world — a support-reply router · Edge cases — a template with the wrong variables	**Doc05 practice (4 of 4)** · Failure — force the parser to actually fail

Week 3

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Doc05 Build Task (part 1 of 2)** · Build it: Reusable Chain Module	**Doc05 Build Task (part 2 of 2)** · Finish and check it against the Test Cases	**Project 6 (bonus)** · LangChainPro (Production LCEL Patterns) — Setup (plan it first: the 7 steps)	**Project 6 (bonus)** · Build How testing works in this project (part 1 of 2)	**Project 6 (bonus)** · Test and finish How testing works in this project (part 2 of 2)	**Project 6 (bonus)** · Build Step 1 — A Classifier That Sends Each Ticket to the Right Reply Chain (part 1 of 2)
Tue	**Project 6 (bonus)** · Test and finish Step 1 (part 2 of 2)	**Project 6 (bonus)** · Build Step 2 — Customer Lookups That Run at the Same Time (part 1 of 2)	**Project 6 (bonus)** · Test and finish Step 2 (part 2 of 2)	**Project 6 (bonus)** · Build Step 3 — Retries, Then a Safe Fallback, When a Model Call Fails (part 1 of 2)	**Project 6 (bonus)** · Test and finish Step 3 (part 2 of 2)	**Project 6 (bonus)** · Build Step 4 — Stream the Reply Live and Build One Structured Record per Ticket (part 1 of 2)
Wed	**Project 6 (bonus)** · Test and finish Step 4 (part 2 of 2), then go through the Checklist Before You Call This Done	**Project 6.1 (practice twin)** · Build CampusDesk — Student-Helpdesk Ticket-Triage Pipeline from its story, then check the solution	**Doc06** · A tool's description is really a prompt	**Doc06** · Checking arguments: Pydantic as the contract	**Doc06** · Choosing between multiple tools	**Doc06** · Getting a tool's failure back to the model, correctly
Thu	**Doc06** · Seeing the whole thing three ways — analogy, trace, and code, side by side	**Doc06** · Parallel tool calls	**Doc06** · Forcing or forbidding tool use (`tool_choice`)	**Doc06** · Chaining tool calls across multiple turns	**Doc06** · Security: tools are a real attack surface	**Doc06** · Prompt injection: when the attack comes from content,

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
						not the user
Fri	**Doc06** · What MCP actually is, and the problem it solves	**Doc06** · MCP's 3 building blocks: Tools, Resources, and Prompts	**Doc06** · MCP servers and clients, and how they actually connect	**Doc06** · Why MCP matters for the multi-agent systems this curriculum builds	**Doc06 practice (1 of 3)** · Basic — your first working tool · Intermediate — watch the model choose between two tools	**Doc06 practice (2 of 3)** · Real-world — a tool backed by a real API call · Edge cases — a missing required argument

Week 4

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Doc06 practice (3 of 3)** · Failure — a crashing tool, and a bad description	**Doc06 Build Task (part 1 of 2)** · Build it: Tool Library	**Doc06 Build Task (part 2 of 2)** · Finish and check it against the Test Cases	**Project 7 (bonus)** · MCPForge (MCP Server) — Setup (plan it first: the 7 steps)	**Project 7 (bonus)** · Build How testing works in this project (part 1 of 2)	**Project 7 (bonus)** · Test and finish How testing works in this project (part 2 of 2)
Tue	**Project 7 (bonus)** · Build Step 1 — A Minimal Server With One Tool (part 1 of 2)	**Project 7 (bonus)** · Test and finish Step 1 (part 2 of 2)	**Project 7 (bonus)** · Build Step 2 — A Real SQLite-Backed Store, and the Search Tool (part 1 of 2)	**Project 7 (bonus)** · Test and finish Step 2 (part 2 of 2)	**Project 7 (bonus)** · Build Step 3 — Add the Resource and the Prompt (part 1 of 2)	**Project 7 (bonus)** · Test and finish Step 3 (part 2 of 2)
Wed	**Project 7 (bonus)** · Build Step 4 — A Real Client That Connects and Uses All	**Project 7 (bonus)** · Test and finish Step 4 (part 2 of 2), then go through the Checklist	**Project 7.1 (practice twin)** · Build LinkKeeper — Personal Bookmarks MCP Server (Tools, Resource & Prompt) from its	**Doc07** · The ReAct loop: think → act → observe	**Doc07** · The scratchpad: how the loop remembers its own steps	**Doc07** · Setting a limit on the number of steps

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
	Three (part 1 of 2)	Before You Call This Done	story, then check the solution			
Thu	**Doc07** · Common single-agent failures, worth naming now	**Doc07** · Agent memory: what actually persists between calls	**Doc07** · Planning before acting	**Doc07** · When NOT to use an agent	**Doc07** · Evaluating an agent, not just a single call	**Doc07 practice (1 of 3)** · Basic — trace the loop on paper first · Intermediate — build the loop yourself
Fri	**Doc07 practice (2 of 3)** · Real-world — compare your loop to the library's · Edge cases — a task that needs no tool at all	**Doc07 practice (3 of 3)** · Failure — watch it loop, then measure the cost of looping at all	**Project 2** · ResearchHand (Worker & Verifier) — Setup (plan it first: the 7 steps)	**Project 2** · Build How testing works in this project (part 1 of 2)	**Project 2** · Test and finish How testing works in this project (part 2 of 2)	**Project 2** · Build Step 1 — A Single LCEL Chain That Answers One Question, No Tools (part 1 of 2)

Week 5

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Project 2** · Test and finish Step 1 (part 2 of 2)	**Project 2** · Build Step 2 — One Tool, One Loop: the Smallest Possible Working Agent (part 1 of 2)	**Project 2** · Test and finish Step 2 (part 2 of 2)	**Project 2** · Build Step 3 — Scaling to 3 Tools With a Hard Step Limit and Failure Recovery (part 1 of 2)	**Project 2** · Test and finish Step 3 (part 2 of 2)	**Project 2** · Build Step 4 — A Verifier Agent That Catches the Worker's Bad Answers (final: 2 agents) (part 1 of 2)
Tue	**Project 2** · Test and finish Step 4 (part 2 of 2), then go through the Checklist	**Project 2.1 (practice twin)** · Build LoanDesk — Tool-Using Finance Assistant	**Project 8 (bonus)** · MCPBridge (MCP Client Agent) — Setup (plan it	**Project 8.1 (practice twin)** · Build OpsPilot — On-Call Assistant Over	**Doc08** · Embeddings: meaning turned into geometry	**Doc08** · Document loaders: getting text out of real files

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
	Before You Call This Done	With a Verifier from its story, then check the solution	first: the 7 steps)	Discovered MCP Tools from its story, then check the solution		
Wed	**Doc08** · Chunking: why where you split matters more than it seems like it should	**Doc08** · Vector stores and searching for the top matches	**Doc08** · "Lost in the middle": more text isn't automatically better	**Doc08** · RAG changes what gets made up — it doesn't remove the problem	**Doc08** · Hybrid search: combining keyword and vector search	**Doc08** · Reranking: a second, more careful pass over the top results
Thu	**Doc08** · Metadata filtering	**Doc08** · Advanced retrieval strategies: when plain top-k isn't enough	**Doc08** · Evaluating a RAG system	**Doc08** · RAG-specific prompt injection: your own knowledge base can attack your own agent	**Doc08** · Exposing your retriever as an MCP tool	**Doc08 practice (1 of 3)** · Basic — see embeddings as geometry, not theory · Intermediate — chunking method changes the answer
Fri	**Doc08 practice (2 of 3)** · Real-world — build and search a real knowledge base · Edge cases — when the answer spans two chunks	**Doc08 practice (3 of 3)** · Failure — a bad split, and a `k` comparison	**Doc08 Build Task (part 1 of 2)** · Build it: Retriever Module	**Doc08 Build Task (part 2 of 2)** · Finish and check it against the Test Cases	**Project 9 (bonus)** · PromptShield (Injection-Hardened RAG Agent) — Setup (plan it first: the 7 steps)	**Project 9.1 (practice twin)** · Build ReviewGuard — Injection-Hardened Product-Review Summarizer from its story, then check the solution

Week 6

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Doc08b** · The event loop: one thread, many waiting tasks	**Doc08b** · Why async helps waiting-on-something work, and does nothing for heavy-computing work	**Doc08b** · `asyncio.gather`: running things at the same time instead of one after another	**Doc08b** · Sync vs. async client: same tool, different way of running	**Doc08b** · Common mistakes, and why they're sneaky	**Doc08b practice (1 of 3)** · Basic — your first coroutine · Intermediate — feel the speed difference
Tue	**Doc08b practice (2 of 3)** · Real-world — convert a real function to async · Edge cases — proving `gather` actually waits	**Doc08b practice (3 of 3)** · Failure — accidentally blocking the event loop	**Doc09** · LangChain vs. LangGraph vs. RAG — how these three actually relate	**Doc09** · Tool-calling inside a graph, and the path from one agent to many	**Doc09** · From a hidden loop to a clear graph	**Doc09** · State: one typed object flowing through the graph
Wed	**Doc09** · Nodes and edges, including conditional edges	**Doc09** · Loops: on purpose, not by accident	**Doc09** · Checkpointers: saved state that survives a pause	**Doc09** · `interrupt()`: pausing for a human	**Doc09** · Time travel and editing a paused run	**Doc09** · Parallel branches (fan-out / fan-in)
Thu	**Doc09** · Subgraphs: a graph as one node in a bigger graph	**Doc09** · Streaming a graph's execution	**Doc09** · Debugging a graph visually	**Doc09** · Long-term memory: remembering across separate conversations	**Doc09 practice (1 of 3)** · Basic — your first graph · Intermediate — real branching	**Doc09 practice (2 of 3)** · Real-world — rebuild Project 2 as a graph · Edge cases — an unhandled routing value

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Fri	**Doc09 practice (3 of 3)** · Failure — an endless loop, and a real pause/resume	**Doc09 Build Task (part 1 of 2)** · Build it: Graph Skeleton (feeds into Project 3)	**Doc09 Build Task (part 2 of 2)** · Finish and check it against the Test Cases	**Project 10 (bonus)** · MemoryKeeper (Persistent Cross-Session Memory) — Setup (plan it first: the 7 steps)	**Project 10.1 (practice twin)** · Build FitPal — Persistent Fitness Coach With Cross-Session Memory from its story, then check the solution	**Doc10** · Search as a routed step, not something that always runs

Week 7

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Doc10** · Why "agentic RAG" is really a graph problem, not a search problem	**Doc10** · State must carry what was found, not just what was asked	**Doc10** · Multi-step search: when one search isn't enough	**Doc10** · Combining retrieval with tool calls	**Doc10** · Caching search results within one run	**Doc10** · When human approval should block vs. just notify
Tue	**Doc10 practice (1 of 3)** · Basic — plug search in as a plain tool first · Intermediate — make search conditional	**Doc10 practice (2 of 3)** · Real-world — a real approval pause · Edge cases — search comes back empty	**Doc10 practice (3 of 3)** · Failure — search fails mid-run	**Project 3** · DocuMind (Retriever, Reasoner & Approval) — Setup (plan it first: the 7 steps)	**Project 3** · Build How testing works in this project (part 1 of 2)	**Project 3** · Test and finish How testing works in this project (part 2 of 2)
Wed	**Project 3** · Build Step 1 — A 2-Node Graph With No Branching, Just to Prove the	**Project 3** · Test and finish Step 1 (part 2 of 2)	**Project 3** · Build Step 2 — Saved State and a Human Pause Before	**Project 3** · Test and finish Step 2 (part 2 of 2)	**Project 3** · Build Step 3 — A Retriever Agent That Judges Its Own Search Results (part 1 of 2)	**Project 3** · Test and finish Step 3 (part 2 of 2)

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
	Mechanics (part 1 of 2)		Anything Goes Out (part 1 of 2)			
Thu	**Project 3** · Build Step 4 — A Reasoner Agent That Only Answers From What the Retriever Found (part 1 of 2)	**Project 3** · Test and finish Step 4 (part 2 of 2)	**Project 3** · Build Step 5 — An Approval Agent That Only Asks a Human When It Really Needs To (final: 3 agents) (part 1 of 2)	**Project 3** · Test and finish Step 5 (part 2 of 2), then go through the Checklist Before You Call This Done	**Project 3.1 (practice twin)** · Build PolicyPal — HR Policy Assistant With Citations & Approval from its story, then check the solution	**Doc11** · The real question: one agent, or many?
Fri	**Doc11** · Every pattern below: what / why / when / trade-off	**Doc11** · The `Command` tool: today's way of handing work between agents	**Doc11** · Shared state design: what goes in the shared state, and what doesn't	**Doc11** · Logging across multiple agents: which agent logged what	**Doc11** · Error propagation between agents	**Doc11** · Cost and latency: multi-agent is not free

Week 8

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Doc11** · Agent-to-agent communication patterns beyond the `Command` tool	**Doc11 practice (1 of 3)** · Basic — design on paper before touching code · Intermediate — measure the sequential version	**Doc11 practice (2 of 3)** · Real-world — build and compare the supervisor version · Edge cases — an ambiguous routing decision	**Doc11 practice (3 of 3)** · Failure — a loop that won't converge, and a cost-cutting pass	**Project 4** · ContentForge (Supervisor-Led Team) — Setup (plan it first: the 7 steps)	**Project 4** · Build How testing works in this project (part 1 of 2)
Tue	**Project 4** · Test and finish How testing works in this project (part 2 of 2)	**Project 4** · Build Step 1 — One Specialist's Drafting Logic, Working Alone, No Agent Behavior (part 1 of 2)	**Project 4** · Test and finish Step 1 (part 2 of 2)	**Project 4** · Build Step 2 — That Specialist, Now a Real Agent That Can Improve Its Own Work (part 1 of 2)	**Project 4** · Test and finish Step 2 (part 2 of 2)	**Project 4** · Build Step 3 — A Research Agent That Looks Things Up Before Anyone Else Works (part 1 of 2)
Wed	**Project 4** · Test and finish Step 3 (part 2 of 2)	**Project 4** · Build Step 4 — An Analysis Agent That Turns Raw Research Into Organized Findings (part 1 of 2)	**Project 4** · Test and finish Step 4 (part 2 of 2)	**Project 4** · Build Step 5 — The Full Team: a Supervisor Routing 4 Specialists via Command (final: 5 agents) (part 1 of 2)	**Project 4** · Test and finish Step 5 (part 2 of 2), then go through the Checklist Before You Call This Done	**Project 4.1 (practice twin)** · Build TripCraft — Multi-Agent Travel Itinerary Generator from its story, then check the solution

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Thu	**Project 11 (bonus)** · MCPCrew (Multi-Agent Research Team) — Setup (plan it first: the 7 steps)	**Project 11.1 (practice twin)** · Build ClientLens — Multi-Agent Customer-360 Team Over MCP Services from its story, then check the solution	**Project 12 (bonus)** · MCPResearch (Agentic Pipeline as an MCP Tool) — Setup (plan it first: the 7 steps)	**Project 12.1 (practice twin)** · Build ComplyDesk — Policy-Checked Support Replies Behind One MCP Tool from its story, then check the solution	**Doc12** · FastAPI: request/response shapes are Pydantic again, one layer up	**Doc12** · Database storage: what to save, and why start with SQLite
Fri	**Doc12** · Docker: what a container actually is, and why it exists	**Doc12** · Caching: what's safe, and what isn't	**Doc12 practice (1 of 3)** · Basic — your first FastAPI route · Intermediate — wrap real logic behind a route	**Doc12 practice (2 of 3)** · Real-world — persist and read back real data · Edge cases — a request that fails validation	**Doc12 practice (3 of 3)** · Failure — a dead connection, and a real container	**Doc12 Build Task (part 1 of 2)** · Build it: Project 4 as an API

Week 9

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Doc12 Build Task (part 2 of 2)** · Finish and check it against the Test Cases	**Doc13** · Why testing something that isn't always the same needs a different mindset	**Doc13** · The testing layers, and what each one actually checks	**Doc13** · Regression testing for prompts	**Doc13** · Test sets and LLM-as-a-judge	**Doc13** · Watching your system: what to record for each run
Tue	**Doc13 practice (1 of 3)** · Basic — a normal, exact test ·	**Doc13 practice (2 of 3)** · Real-world — score an	**Doc13 practice (3 of 3)** · Failure — prove the suite actually	**Doc13 Build Task (part 1 of 2)** · Build it: Test Suite for Project 4	**Doc13 Build Task (part 2 of 2)** · Finish and check it	**Doc14** · Why "real cause" and "just the symptom" are

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
	Intermediate — test a tool without calling the LLM	answer that isn't exact · Edge cases — fix a flaky test	catches a regression		against the Test Cases	different things
Wed	**Doc14** · Reading an error trace as a map, not noise	**Doc14** · Reproduce before you guess	**Doc14** · Debugging systems with many layers	**Doc14 practice (1 of 4)** · Python · API	**Doc14 practice (2 of 4)** · OpenAI · LangChain	**Doc14 practice (3 of 4)** · RAG · LangGraph
Thu	**Doc14 practice (4 of 4)** · Multi-agent	**Doc15** · Read: Project Index	**Doc15 practice (1 of 1)** · Basic — run the 7-step process on a mini scenario	**Doc16** · Unclear requirements are the real test	**Doc16** · Requirements → architecture is matching, not inspiration	**Doc16** · Non-functional requirements shape the design just as much as functional ones
Fri	**Doc16** · Defending a design when it's pushed on	**Doc16 practice (1 of 2)** · Basic — a simple single-agent design · Intermediate — a multi-agent pipeline design	**Doc16 practice (2 of 2)** · Real-world — unclear, conflicting requirements	**Doc17** · Read: The Four-Depth Format (used for every question)	**Doc17** · Read: Question Coverage (per document you've already finished): Document 01 — Python Foundations	**Doc17** · Read: Question Coverage (per document you've already finished): Document 08 — RAG

Week 10

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Doc17** · Read: Question Coverage (per document you've already finished): Document 11	**Doc17 practice (1 of 2)** · Basic — cold recall · Intermediate — apply the follow-up	**Doc17 practice (2 of 2)** · Real-world — a cold interview on your own project	**Doc18** · Read: What You'll Be Given (not designed for you)	**Doc18** · Read: What You'll Design and Build	**Doc18 practice (1 of 2)** · Basic — critique your own Stage-1 design before you build · Intermediate — defend one

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
	— Multi-Agent Systems	template yourself				non-obvious trade-off under pushback
Tue	**Doc18 practice (2 of 2)** · Real-world — a cold design-review interview on your finished capstone	**Doc19** · MLOps vs. LLMOps: what's actually different here	**Doc19** · Versions for prompts/settings: treating them as real, tracked things	**Doc19** · CI/CD for agent systems: the test suite as a release gate	**Doc19** · Ways to release: canary, blue-green, and shadow	**Doc19** · Watching for slow decline: cost, speed, and quality over time
Wed	**Doc19** · Rolling back: the real safety net	**Doc19** · Backup plans if your provider has an outage	**Doc19 practice (1 of 3)** · Basic — a simple version log · Intermediate — build the gate	**Doc19 practice (2 of 3)** · Real-world — act out a canary release · Edge cases — a bad release that slips past the gate	**Doc19 practice (3 of 3)** · Failure — a real rollback trigger	**Project 5** · ContentForge Pro (Production Platform) — Setup (plan it first: the 7 steps)
Thu	**Project 5** · Build How testing works in this project (part 1 of 2)	**Project 5** · Test and finish How testing works in this project (part 2 of 2)	**Project 5** · Build Step 1 — ContentForge, Reachable Over HTTP for the First Time (part 1 of 2)	**Project 5** · Test and finish Step 1 (part 2 of 2)	**Project 5** · Build Step 2 — Every Run Saved to a Database and Findable by ID (part 1 of 2)	**Project 5** · Test and finish Step 2 (part 2 of 2)
Fri	**Project 5** · Build Step 3 — Containerized and Rate-Limited: Runs the Same	**Project 5** · Test and finish Step 3 (part 2 of 2)	**Project 5** · Build Step 4 — A Test Suite That Catches a Bad Change Before It Ships (final) (part 1 of 2)	**Project 5** · Test and finish Step 4 (part 2 of 2), then go	**Project 5.1 (practice twin)** · Build TripCraft Pro — Production	**Project 13 (bonus)** · CodeGuard (PR-Review Team) — Setup (plan it

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
	Everywhere, Survives Abuse (part 1 of 2)			through the Checklist Before You Call This Done	Itinerary Service (API, Database, Docker, Evals) from its story, then check the solution	first: the 7 steps)

Week 11

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Mon	**Project 13.1 (practice twin)** · Build DispatchGuard — Multi-Agent Outgoing-Message Review in One Pass from its story, then check the solution	**Doc20** · The web-layer pattern: input → your pipeline → output	**Doc20** · Streamlit: the fastest path to a UI	**Doc20** · Flask: a minimal, explicit web framework	**Doc20** · FastAPI: typed async APIs (recap from Doc12)	**Doc20** · Django: batteries-included, for products
Tue	**Doc20** · Choosing the right tool	**Doc20** · Which framework for YOUR AI project — why, and how	**Doc20** · File uploads: the key to document AI apps	**Doc20** · Streaming the answer to the UI	**Doc20** · A real HTML + JavaScript frontend calling your API	**Doc20** · Deploying your web app

Day	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
Wed	**Doc20 practice (1 of 4)** · Basic — a Streamlit UI · Intermediate — a Flask app with a form	**Doc20 practice (2 of 4)** · Real-world — a FastAPI API + a form page · Edge — a Django view	**Doc20 practice (3 of 4)** · Failure/choice — pick the right tool · Intermediate — a file-upload app	**Doc20 practice (4 of 4)** · Intermediate — stream the answer · Intermediate — an HTML + JavaScript frontend	**Doc20 Build Task (part 1 of 2)** · Build it: Put a UI on a Real Pipeline	**Doc20 Build Task (part 2 of 2)** · Finish and check it against the requirements